

System Requirements Document (SRD) and Operations Manual

DILBOT Laboratory Automation System

Project: DILBOT (Devices and Interface Lab's
Design and Development of Robotic System)

December 02, 2025

Document Control

Version	Date	Author(s)	Description of Changes
1.0	December 02, 2025	Project Lead	Initial draft of the SRD.

This document specifies the functional and non-functional requirements for the DILBOT Laboratory Automation System. It also serves as a high-level operations and integration manual.

Contents

Contents	3
List of Figures	4
1 Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions, Acronyms, and Abbreviations	5
1.4 References	6
2 Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	7
2.3 User Classes and Characteristics	7
2.4 Operating Environment	8
2.5 General Constraints	8
2.6 Use Cases	9
2.6.1 Use Case: Automated Perovskite Precursor Deposition	9
3 Functional Requirements	10
3.1 Robot Arm (Mitsubishi Electric RH-6CH6020-D-S15)	10
3.1.1 Robot Arm - Installation and Operation	11
3.2 Spin Coater (SCS G4 Spin Coater Series)	13
3.3 Implementation	13
3.4 Hardware Connection	13
3.5 Identify the Serial Port	14
3.6 Grant Port Access Permission	14
3.7 Navigate to Project Directory	14
3.8 Execute the Python Script	14
3.9 Program Behavior	14
3.10 Pipetting Module (Hamilton ZEUS X1)	14
3.10.1 Hardware Setup	15
3.10.2 STM32CubeMX Configuration	15
3.10.3 The Protocol Logic	16
3.10.4 Software Implementation	16
3.10.5 Application Flow	17
3.11 Custom Gripper	18
3.11.1 Hardware Architecture	18
3.11.2 Control Logic	18
3.11.3 Debounce and Trigger Handling	19
3.11.4 Actuation Parameters	19
3.11.5 Code for OpenRB-150	19
3.11.6 Electrical and Communication Interface	21
3.11.7 Pin Mapping	21
3.11.8 Summary	21
3.11.9 Custom Gripper - Installation and Operation	21

3.12	Camera - Installation and Operation	22
3.13	System Integration & Recipe Management	23
3.13.1	I/O pin distribution for both robots	23
3.14	User Interface (UI)	25
4	Non-Functional Requirements	26
4.1	Performance Requirements	26
4.2	Safety Requirements	26
4.3	Reliability Requirements	26
4.4	Maintainability Requirements	27
4.5	Usability Requirements	27
4.6	Security Requirements	27
5	System Architecture (High-Level)	28
5.1	Software Architecture	28
5.2	Hardware Architecture	28
6	Verification Requirements	30
A	Appendices	31
A.1	Appendix A: Detailed Communication Protocols	31
A.2	Appendix B: CAD Drawings and Mechanical Schematics	31
A.3	Appendix C: Electrical Schematics	31
A.4	Appendix D: Test Plans	31
A.5	Appendix E: Glossary	32

List of Figures

1	Robot specification	12
2	Integration with Raspberry Pi	13
3	Gripper motor interface and trigger circuit.	21
4	Pin mapping for the OpenRB-150 and peripheral components.	21
5	High-level diagram of the DILBOT hardware components.	29
6	Example CAD drawing of the integrated system.	31

1 Introduction

1.1 Purpose

This System Requirements Document (SRD) specifies the functional and non-functional requirements for the DILBOT Laboratory Automation System. This system aims to integrate a Robot Arm, Spin Coater, Pipetting Module, a custom-designed gripper, UV ozone cleaner, nitrogen flushing area and other complimentary devices/equipment into a cohesive automated platform for advanced laboratory processes, specifically for material and device acceleration in emerging photovoltaics technologies.

1.2 Scope

The scope of this document covers the integrated control software, mechanical interfaces, electrical connections, and operational procedures for the automated system. It includes requirements for:

- Robot arm control and path planning/motion planning.
- Spin coater operation and recipe management.
- Pipetting module control and liquid handling.
- Custom gripper design, fabrication, and control.
- Control and operation of UV ozone cleaner, flushing station, heating tray and other complimentary equipment.
- Floor planning and management.
- Inter-module communication and synchronization.
- User interface for recipe creation and execution.
- Safety and error handling mechanisms.

Exclusions: This document does not cover the detailed design of individual components (e.g., internal mechanics of the robot arm or spin coater), nor does it cover the specific chemical processes or material science recipes beyond their programmatic control.

1.3 Definitions, Acronyms, and Abbreviations

DILBOT Devices and Interface Lab's Design and Development of AI assisted Robotic System

SRD System Requirements Document

UI User Interface

API Application Programming Interface

CAD Computer-Aided Design

PCB Printed Circuit Board

RPM Revolutions Per Minute

LLD Liquid Level Detection (Hamilton Pipette)

HMI Human-Machine Interface (often used interchangeably with UI in industrial contexts)

GUI Graphical User Interface

TCP Tool Center Point (Robot Arm)

1.4 References

1. Mitsubishi Industrial Robot RH-3CH-Sxx/RH-6CH-Sxx INSTRUCTION MANUAL (RH_6CH.pdf)
2. SCS G4 Spin Coater Series 0725 (43 SCS G4 Spin Coater Series 0725.pdf)
3. ZEUS X1 Integrator Manual (7624482_Manual_ZEUS_X_Pipetting_A5_COREII_R21.pdf)
4. ZEUS X1 - instructions (Instructions_ZEUS X1.pdf)
5. Project Proposal: Drona–Appendix_DILBOT_27-06.docx
6. Existing CAD models of the laboratory setup.
7. [**TO BE SPECIFIED:** Relevant industry safety standards, e.g., ISO 10218-1/2 for industrial robots, IEC 61010 for lab equipment][In consultation with Devices and Interfaces laboratory team, some standard operating procedures will be created to carry out the experiments in safe manners]

2 Overall Description

2.1 Product Perspective

The DILBOT system will be a standalone automated workstation integrated into a laboratory environment. It will receive high-level recipe commands from a user and execute them by orchestrating the actions of the robot arm, spin coater, pipette, hot plates, nitrogen flushing area and custom gripper. The system will provide feedback on its status and allow for manual intervention if necessary. The system will integrate with existing laboratory infrastructure and be capable of operating autonomously for defined periods in a controlled environment.

2.2 Product Functions

The primary functions of the DILBOT system include:

- Automated handling and transfer of substrates/samples between modules using the robot arm and custom gripper.
- Precise application of liquid materials via the pipette, including aspiration, dispensing, and tip management (two dispensing methods to be used).
- Controlled spin coating of substrates with programmable speed, ramp, and dwell times.
- Automated loading and unloading of consumables (e.g., pipette tips, well plates, substrates) from designated locations.
- Execution of complex, multi-step laboratory recipes defined by the user.
- Real-time monitoring of system status, progress, and error reporting.
- Logging of all operational data, sensor readings, and error events for traceability and debugging.
- Emergency stop and safety interlock mechanisms to ensure operator safety.

2.3 User Classes and Characteristics

Laboratory Researchers/Scientists: • **Characteristics:** Possess scientific domain knowledge, may have limited programming experience.

- **Needs:** Intuitive GUI for recipe creation and execution, clear status feedback, reliable and repeatable operation, easy data logging access.

Maintenance Personnel: • **Characteristics:** Possess basic mechanical and electrical troubleshooting skills.

- **Needs:** Access to diagnostic tools, calibration procedures, detailed technical documentation, easy access to replaceable components.

Developers/Engineers (Team Members): • **Characteristics:** Possess strong programming, mechanical, and electrical engineering skills.

- **Needs:** Access to source code, APIs, detailed system architecture documentation, debugging interfaces, ability to extend functionality.

2.4 Operating Environment

The system will operate in a controlled laboratory environment, potentially within a fume hood or glove box for certain processes.

- **Temperature:** To be specified
- **Humidity:** To be specified
- **Power Supply:** To be specified
- **Connectivity:** To be specified
- **Vibration:** To be specified
- **Cleanliness:** To be specified

2.5 General Constraints

- **Safety:** The system must comply with all relevant laboratory safety standards (e.g., chemical handling [to be given by lab], electrical safety) and robot safety guidelines (e.g., ISO 10218). Emergency stop mechanisms and safety interlocks.
- **Footprint:** The integrated system must fit within the existing laboratory space and adhere to the dimensions specified in the latest CAD model.
- **Modularity:** System components and software modules should be designed for high modularity to allow for independent development, testing, and future upgrades or replacements without requiring major system redesigns.
- **Maintainability:** The system should be designed for ease of maintenance, troubleshooting, and calibration. This includes clear labeling, accessible components, and comprehensive documentation.
- **Data Integrity:** All experimental data, recipe parameters, and system logs must be stored reliably and be easily retrievable.
- **Software Language:** Primary control software to be developed in [TO BE SPECIFIED: Preferred programming language, e.g., Python, C++].
- **CAD Software:** All mechanical designs to be created using SolidWorks and Fusion 360, and FEA and other checks will be done on custom components before final hand-off.

2.6 Use Cases

2.6.1 Use Case: Automated Perovskite Precursor Deposition

Actor: Laboratory Researcher

Preconditions: • System is powered on and initialized.

- Robot arm is homed.
- Spin coater chuck is clean and ready.
- Pipette tips are loaded in the tip rack.
- Precursor solution is in a designated well plate.
- Substrates are loaded in a substrate holder accessible by the robot.

Flow of Events: 1. Researcher selects "Perovskite Precursor Deposition (example)" recipe from the UI.

2. System verifies all preconditions.
3. Robot arm picks up a fresh substrate from the holder using the custom gripper.
4. Robot arm transfers the substrate to the spin coater chuck and places it securely.
5. Spin coater program "Precursor_Spin" is initiated by the central control system.
6. While spin coater is ramping up, robot arm moves to the pipette tip rack.
7. Robot arm picks up a pipette tip.
8. Robot arm moves the Hamilton pipette to the precursor solution well plate.
9. Hamilton pipette aspirates [TO BE SPECIFIED: Volume] of precursor solution using LLD.
10. Robot arm moves the Hamilton pipette over the center of the spinning substrate.
11. Hamilton pipette dispenses the precursor solution onto the substrate.
12. Spin coater completes its program (including ramp-down and dwell times).
13. Another fluid is dispensed.
14. Hamilton pipette moves to the waste bin and ejects the used tip.
15. Robot arm picks up the coated substrate from the spin coater.
16. Robot arm transfers the coated substrate to a designated drying rack.
17. System logs successful completion and relevant parameters.

Postconditions: • Substrate is coated and transferred to the drying rack.
 • System is ready for the next operation.

Possible Error Conditions: • Substrate drop during transfer -> Robot stops, alerts user, logs error.
 • Liquid level too low in well plate -> Pipette stops, alerts user, logs error.
 • Spin coater motor fault -> Spin coater stops, alerts user, logs error.
 • Communication loss with any module -> System pauses, attempts re-establishment, alerts user.

3 Functional Requirements

This section details the functional requirements for each major component of the DILBOT system.

3.1 Robot Arm (Mitsubishi Electric RH-6CH6020-D-S15)

- Controller series CR800-CHD-S15
- Load capacity 6kg
- Reach 600mm
- Mass 17kg
- Serial number of robot 1: BP5040019R
- Date: 2025-04
- Motor power of joint: J1, J2: 200W ; J3, J4: 100W

FR-RA-001 Communication Establishment: The system shall be able to establish and maintain a persistent communication link with the Mitsubishi Electric Robot Arm controller via Ethernet (TCP/IP).

- **Sub-FR-RA-001.1:** The system shall support automatic reconnection attempts in case of communication loss.
- **Sub-FR-RA-001.2:** The system shall provide a clear indication of the robot arm's connection status on the UI.

FR-RA-002 Precise Movement Control: The system shall be able to send commands to the robot arm for precise movement to specified X, Y, Z coordinates (position) and A, B, C angles (orientation) in Cartesian space.

- **Sub-FR-RA-002.1:** The system shall allow specification of movement speed and acceleration for each motion command.
- **Sub-FR-RA-002.2:** The system shall support linear and interpolated movements between points.

FR-RA-003 Joint and Cartesian Control: The system shall support both joint-space movements and Cartesian-space movements.

FR-RA-004 Status Feedback: The system shall be able to continuously read and display the current TCP position (X, Y, Z, A, B, C) and the robot's configuration flag.

FR-RA-005 Collision Avoidance: The system shall implement a software-based collision avoidance logic using the loaded CAD model of the workspace.

- **Sub-FR-RA-005.1:** The system shall allow definition of exclusion zones and keep-out areas.

- **Sub-FR-RA-005.2:** The system shall provide a warning or halt movement if a potential collision is detected.

FR-RA-006 Teach-in Functionality: The system shall allow users to manually jog the robot arm to desired positions and save them as named waypoints.

- **Sub-FR-RA-006.1:** Teach-in shall be possible via a joystick or software controls on the UI.
- **Sub-FR-RA-006.2:** Saved waypoints shall include position, orientation, and configuration flags.

FR-RA-007 Tool Management: The system shall support defining and switching between multiple tools by updating the robot's TCP offset.

3.1.1 Robot Arm - Installation and Operation

Installation Guide

Robot installation requires securing it using the four securing points at the bottom. The electric connections can be followed according to the manual **Add the electrical connections diagram**

Network settings

All the devices need to be on the same network to communicate with each other. So change the network and ethernet settings of your PC to **192.168.0.n** where 'n' is the variable. Change subnet mask to 255.255.255.0.

Check the robot IP using the following steps: **Add the steps using teach pendant**

Using RT Toolbox3

Install the **RT Toolbox3 pro** from the official website of https://www.mitsubishielectric.com/fa/download/software/detailsearch.page?mode=software&kisyu=/robot&shiryoid=1000000192&lang=2&select=0&softid=1&infostatus=1_4_2&viewradio=0&viewstatus=&viewpos=

Standard Operating Procedure (SOP)

This SOP outlines the routine procedures for operating the robot arm within the DILBOT system.

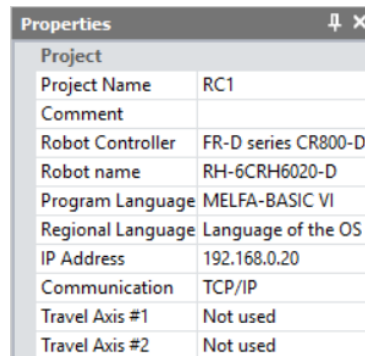
1. System Startup:

- *[PLACEHOLDER: Step-by-step instructions on the correct power-up sequence for the robot arm controller and enabling servo power.]*
- *Launch the RT3 [htbp]*

2. Homing Procedure:

- *[PLACEHOLDER: Instructions for executing the robot's homing sequence, ensuring it starts from a known, calibrated position. Describe both automated homing from the UI and manual procedures if necessary.]*

3. Manual Operation (Jogging):

A screenshot of a 'Properties' dialog box with a title bar containing a pin icon and a close button. The dialog is divided into two panes. The left pane is titled 'Project' and contains a list of properties. The right pane shows the values for these properties.

Project	
Project Name	RC1
Comment	
Robot Controller	FR-D series CR800-D
Robot name	RH-6CRH6020-D
Program Language	MELFA-BASIC VI
Regional Language	Language of the OS
IP Address	192.168.0.20
Communication	TCP/IP
Travel Axis #1	Not used
Travel Axis #2	Not used

Figure 1: Robot specification

- *[PLACEHOLDER: Guide on using the UI's teach-in/jogging controls. Detail how to switch between joint, Cartesian, and tool coordinate systems. Explain speed controls and safety precautions during manual movement.]*

4. Defining and Saving Waypoints:

- *[PLACEHOLDER: How to move the robot to a specific location (e.g., above the spin coater) and save its coordinates and orientation as a named waypoint within the DILBOT software for use in recipes.]*

5. System Shutdown:

- *[PLACEHOLDER: The correct sequence for parking the robot, disabling servo power, and shutting down the controller and software.]*

3.2 Spin Coater (SCS G4 Spin Coater Series)

FR-SC-001 Communication Establishment: The system shall be able to establish and maintain communication with the SCS G4 Spin Coater.

- **Sub-FR-SC-001.1:** [TO BE SPECIFIED: Determine exact communication interface (e.g., RS-232, USB, Ethernet) and protocol].
- **Sub-FR-SC-001.2:** The system shall provide a clear indication of the spin coater's connection status on the UI.

FR-SC-002 Program Execution: The system shall be able to program and execute spin coating cycles consisting of up to 20 steps.

FR-SC-003 Step Parameter Definition: For each step, the system shall allow programmatic definition of: Rotational speed, Ramp time, and Dwell time.

FR-SC-004 Cycle Control: The system shall be able to initiate, pause, resume, and terminate a spin coating cycle programmatically.

FR-SC-005 Status Query: The system shall be able to query the current operational status of the spin coater.

FR-SC-006 Program Storage: The system shall be able to store and recall up to 30 spin coating programs.

3.3 Implementation

3.4 Hardware Connection

Connect the USB dongle to the Raspberry Pi (RPi).

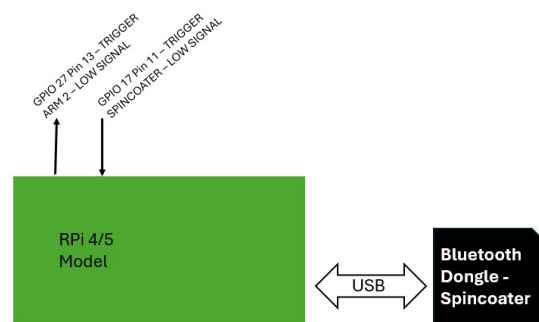


Figure 2: Integration with Raspberry Pi

3.5 Identify the Serial Port

Check the available serial devices using the following command:

```
1 ls /dev/tty*
```

A device such as `ttyUSB0` should appear.

3.6 Grant Port Access Permission

Provide permission to access the detected serial port:

```
1 sudo chmod 777 /dev/ttyUSB0
```

3.7 Navigate to Project Directory

Change the directory to the Desktop:

```
1 cd Desktop
```

3.8 Execute the Python Script

Run the Python script using the command below:

```
1 python3 spincoater_07012025.py
```

3.9 Program Behavior

The script waits for a LOW input signal on GPIO 17 (physical pin 11). Once the signal is detected, the configured recipe is transmitted to the spin coater.

After sending the recipe, the program continuously monitors the execution status. If the process completes successfully, the output status is set to **TRUE**; otherwise, it is set to **FALSE**.

A failsafe mechanism is implemented to track the total expected execution time of each recipe, including all individual steps and an additional buffer time. If the execution exceeds this predefined duration, a stop command is automatically sent to the spin coater to prevent unintended operation.

3.10 Pipetting Module (Hamilton ZEUS X1)

FR-HP-001 Communication Establishment: The system shall be able to establish and maintain communication via USB/Serial connection.

FR-HP-002 Liquid Handling Commands: The system shall be able to send commands for aspiration, dispensing, and mixing cycles.

FR-HP-003 Volume Control: The system shall support precise volume control with a minimum increment of [TO BE SPECIFIED: e.g., 0.1 μ L].

FR-HP-004 Liquid Level Detection (LLD): The system shall integrate and utilize the LLD functionality.

FR-HP-005 Tip Management: The system shall be able to control picking up new tips and ejecting used tips.

FR-HP-006 Deck Geometry Configuration: The system shall allow programmatic configuration of the Hamilton module's deck geometry.

FR-HP-007 Status and Error Reporting: The system shall be able to query the current status and receive specific error codes.

3.10.1 Hardware Setup

1. Components Required:

- **MCU:** STM32F103C8T6 (Bluepill)
- **Transceiver:** MCP2551 (5V Logic)
- **Module:** Hamilton ZEUS X1
- **Power Supply:** 48V DC (Motors), 5V DC (Transceiver)
- **Termination:** 120 Ω resistor.

2. Wiring Connections

STM32 Pin	MCP2551 Pin	Description
PB8	Pin 4 (RXD)	CAN Receive
PB9	Pin 1 (TXD)	CAN Transmit
GND	Pin 2 (GND)	Connect to GND
5V	Pin 3 (VDD)	Connect to +5V

Table 1: Microcontroller to Transceiver Wiring

MCP2551	ZEUS X1 Module
Pin 7 (CANH)	Pin 1 (CAN H)
Pin 6 (CANL)	Pin 2 (CAN L)

Table 2: CAN Bus Wiring

3.10.2 STM32CubeMX Configuration

To replicate the project from scratch, configure the `.ioc` file as follows:

1. Clock Configuration:

- **High Speed Clock (HSE):** Crystal/Ceramic Resonator.
- **System Clock:** 72 MHz.
- **APB1 Peripheral Clock:** 36 MHz (Critical for CAN baud rate).

2. CAN Settings:

- **Mode:** Normal
 - **Prescaler:** 4
 - **Time Quanta in Bit Seg 1:** 15 Times
 - **Time Quanta in Bit Seg 2:** 2 Times
 - **Rejection Jump Width:** 1 Time
 - **Baud Rate Calculation:** $36MHz/4/(1 + 15 + 2) = 500$ kbps.
3. **Interrupts (NVIC):** Enable **CAN RX interrupts** (optional) and specifically:
- **CAN RX1 Interrupt:** Enabled (with FIFO1).

3.10.3 The Protocol Logic

The ZEUS X1 does not use standard data packets. It requires a specific handshake (Kick) and control bytes.

1. **CAN IDs (11-bit Standard):** Assuming Master ID = 0 and Zeus Node ID = 1:
 - **0x401 (ZEUS_ID_KICK):** Sent by Master to wake up Slave.
 - **0x001 (ZEUS_ID_DATA):** Sent by Master containing command string.
 - **0x020 (ZEUS_ID_RESPONSE):** Sent by Slave to Master.
 - **0x420 (ZEUS_RECEIVE_KICK_ID):** Sent by Slave to Master (Kick) when sending multi-frame responses.
2. **Transmission Sequence:** Every command follows this sequence:
 - (a) **Kick:** Master sends ID 0x401 (DLC 8, Payload 0).
 - (b) **Remote:** Slave responds with a Remote Frame (RTR).
 - (c) **Data:** Master sends ID 0x001 with payload.
3. **The Control Byte:** The 8th byte (Index 7) of every Data Frame is the Control Byte. It is calculated as:

```

1 // EOM: End of Message (Bit 7)
2 // LEN: Length of data 0-7 (Bits 4-6)
3 // CNT: Frame Counter 1-15 (Bits 0-3)
4 TxData[7] = (EOM << 7) | (LEN << 4) | (COUNTER & 0x0F);
5 // Example for "RF" (Length 2, single frame): 0x80 | 0x20 | 0x01 =
  0xA1

```

3.10.4 Software Implementation

1. **Command Transmission (Fragmented):** The function `Zeus_SendCommand_Generic` handles strings longer than 7 bytes by breaking them into chunks.

```

1 // Key Logic for Fragmentation
2 while (sent < cmd_len) {
3     // 1. Calculate Chunk Size (Max 7 bytes)
4     uint8_t chunk_size = (cmd_len - sent > 7) ? 7 : (cmd_len - sent
5 );

```



```

5
6 // 2. Determine EOM (End of Message)
7 uint8_t is_last = (sent + chunk_size >= cmd_len) ? 1 : 0;
8
9 // 3. Construct Control Byte
10 data_frame[7] = (is_last ? 0x80 : 0x00) | (chunk_size << 4) | (
    frame_counter & 0x0F);
11
12 // 4. Send Data
13 HAL_CAN_AddTxMessage(&hcan, &TxHeader, data_frame, &TxMailbox);
14
15 // 5. If not last, wait for handshake before sending next chunk
16 if (!is_last) {
17     Zeus_WaitForRemote(REMOTE_TIMEOUT_MS);
18 }
19 }

```

2. **Receiving Data (Interrupt Callback):** The `HAL_CAN_RxFifo1MsgPendingCallback` handles the complex state machine of receiving data from the Zeus.

- ****Handshake:**** When Zeus sends a Kick (ID 0x420), the ISR immediately replies with a Remote Frame to ID 0x020 to request the data.
- ****Data Reconstruction:**** When data arrives (ID 0x020), the ISR extracts the length from the control byte and appends payload to `rx_buffer`.
- ****Multi-frame Support:**** If the EOM bit is 0, the ISR automatically sends another Remote Frame to request the next packet.

3.10.5 Application Flow

The `main()` function implements the following initialization routine:

1. **RF (Request Firmware):** Checks if device is connected. LED blinks 3 times on success.
 2. **ZI (Zero Instrument):** Homes the Z-Axis (Vertical).
 3. **DI (Dosing Initialize):** Homes the Pipetting drive.
1. **Operation Loop** After initialization, the system enters an infinite loop waiting for inputs:
- **Aspirate (GPIO Trigger):** If `Aspirate_command_received` flag is set (via external interrupt), it sends:
 - GZgy1000 (Set Gain/Speed).
 - SAai01000 (Smart Aspirate 1000 steps).
 - **Dispose (Button PB7):** If PB7 is pressed:
 - Moves Z-axis.
 - SDdi01000 (Smart Dispense).
 - GUtt05go00 (Tip Eject/Dispose).

3.11 Custom Gripper

The gripper end-effector of the system is actuated using a **Dynamixel AX-12A** smart servo motor, selected for its compact form factor, integrated gearbox, position feedback capability, and serial communication support. The controller used for interfacing and commanding the actuator is an **OpenRB-150** microcontroller, which is compatible with the Dynamixel ecosystem and supports UART-based half-duplex communication without external bus routing components.

3.11.1 Hardware Architecture

The OpenRB-150 communicates with the AX-12A motor using the Dynamixel protocol (version 1.0) over the default serial bus. The motor is mechanically integrated into the gripper to achieve opening and closing actions through angular position commands.

A digital input signal at 24V is used as a trigger to determine the actuation state of the gripper. The signal is isolated and level-shifted before interfacing with the controller. When activated, the motor rotates to a predefined angular setpoint corresponding to the closed state; otherwise, it returns to the default open configuration.

Components:

- Dynamixel AX-12A smart servo motor
- OpenRB-150 microcontroller
- 24V trigger signal source
- Signal conditioning circuit (level shifting and protection)

3.11.2 Control Logic

The gripper control strategy is based on a **two-position actuation scheme** with smooth transition using incremental motion steps. The motor operates in *position control mode*, enabled by the onboard control table of the AX-12A.

A high-level description of the implemented logic is as follows:

1. Initialize the serial bus at 1 Mbps and verify that the Dynamixel actuator is detected.
2. Set the actuator operating mode to position control and apply torque.
3. Configure the base rotation speed through the control table.
4. Continuously monitor the digital trigger signal.
5. Apply software-based debounce to reject spurious transitions.
6. When the trigger is ON, gradually drive the gripper to a closed position (0° – 3°).
7. When the trigger is OFF, gradually drive the gripper to an open position (up to 25°).
8. During motion, query the motor's current position for monitoring.

The system implements a **soft-motion function**, where the target angle is approached in small increments with fixed temporal spacing. This reduces jerk, minimizes mechanical stress, and provides observable motion for debugging.

3.11.3 Debounce and Trigger Handling

The input trigger signal is subject to electrical noise due to its switching characteristics. To ensure reliable actuation, a timing-based debounce filter is applied before evaluating state transitions.

Let $t_d = 100$ ms denote the debounce period. A state change is acknowledged only if the input remains stable for longer than t_d , preventing rapid oscillations in motor commands.

3.11.4 Actuation Parameters

The main operating parameters of the AX-12A are shown in Table 3.

Parameter	Value
Protocol Version	1.0
Baud Rate	1 Mbps
Operating Mode	Position Control
Base Speed (Moving Speed)	100 (0–1023 range)
Closed Position Angle	0°–3°
Open Position Angle	25°
Step Size	0.5°
Step Delay	30 ms

Table 3: Dynamixel AX-12A actuation parameters.

3.11.5 Code for OpenRB-150

Add the code below in arduino IDE compiler and add openRB-150 board in that IDE

```

1  #include <Dynamixel2Arduino.h>
2  const int triggerPin = 2; // Digital Pin D2
3
4  #define DXL_SERIAL Serial1
5  #define DEBUG_SERIAL Serial
6  const int DXL_DIR_PIN = -1; // Not needed for OpenRB
7
8  const uint8_t DXL_ID = 1;
9  const float DXL_PROTOCOL_VERSION = 1.0;
10
11 Dynamixel2Arduino dxl(DXL_SERIAL, DXL_DIR_PIN);
12 using namespace ControlTableItem;
13
14 // === Debounce variables ===
15 int lastTriggerState = HIGH;
16 int stableTriggerState = HIGH;
17 unsigned long lastDebounceTime = 0;
18 const unsigned long debounceDelay = 100; // 100 ms debounce time
19
20 void setup() {
21     DEBUG_SERIAL.begin(115200);
22     //while (!DEBUG_SERIAL);
23
24     dxl.begin(1000000); // Default baud rate for AX-12A
25     dxl.setPortProtocolVersion(DXL_PROTOCOL_VERSION);

```

```

26
27  if (dxl.ping(DXL_ID)) {
28      DEBUG_SERIAL.println("Dynamixel connected!");
29  } else {
30      DEBUG_SERIAL.println("Failed to connect!");
31  }
32
33  dxl.torqueOff(DXL_ID);
34  dxl.setOperatingMode(DXL_ID, OP_POSITION);
35  dxl.torqueOn(DXL_ID);
36
37  dxl.writeControlTableItem(MOVING_SPEED, DXL_ID, 100);  // Base speed
38  (0 1023 )
39  pinMode(triggerPin, INPUT);
40
41  /**
42   * Function to move the motor slowly between two angles
43   * @param start_deg - starting position in degrees
44   * @param end_deg   - target position in degrees
45   * @param step_deg  - step increment per move (smaller = smoother)
46   * @param delay_ms  - delay between steps (higher = slower)
47   */
48  void moveSlow(float start_deg, float end_deg, float step_deg, int
    delay_ms) {
49      if (start_deg < end_deg) {
50          for (float pos = start_deg; pos <= end_deg; pos += step_deg) {
51              dxl.setGoalPosition(DXL_ID, pos, UNIT_DEGREE);
52              float current_position = dxl.getPresentPosition(DXL_ID,
UNIT_DEGREE);
53              DEBUG_SERIAL.print("Current Position: ");
54              DEBUG_SERIAL.println(current_position);
55              delay(delay_ms);
56          }
57      } else {
58          for (float pos = start_deg; pos >= end_deg; pos -= step_deg) {
59              dxl.setGoalPosition(DXL_ID, pos, UNIT_DEGREE);
60              float current_position = dxl.getPresentPosition(DXL_ID,
UNIT_DEGREE);
61              DEBUG_SERIAL.print("Current Position: ");
62              DEBUG_SERIAL.println(current_position);
63              delay(delay_ms);
64          }
65      }
66  }
67
68  void loop() {
69      int reading = digitalRead(triggerPin);
70
71      // Debounce logic
72      if (reading != lastTriggerState) {
73          lastDebounceTime = millis();
74      }
75
76      if ((millis() - lastDebounceTime) > debounceDelay) {
77          if (reading != stableTriggerState) {
78              stableTriggerState = reading;
79

```

```

80     float current_position = dxl.getPresentPosition(DXL_ID,
UNIT_DEGREE);
81
82     if (stableTriggerState == LOW) {
83         // This block runs if the 24V trigger is ON.
84         Serial.println("Trigger is ON. Moving to 0degree...");
85         moveSlow(current_position, 3.0, 0.5, 30);
86     } else {
87         // This block runs if the 24V trigger is OFF.
88         Serial.println("Trigger is OFF. Moving to 25degree...");
89         moveSlow(current_position, 25.0, 0.5, 30);
90     }
91 }
92 }
93
94 lastTriggerState = reading;
95 delay(50); // small delay before next read
96 }

```

3.11.6 Electrical and Communication Interface

Figure 3 illustrates the circuit used to interface the AX-12A motor, OpenRB-150 micro-controller, and the 24V trigger input. The figure includes power routing, signal conditioning, and communications wiring.

Placeholder for Circuit Diagram

Figure 3: Gripper motor interface and trigger circuit.

3.11.7 Pin Mapping

A logical mapping of controller pins, actuator power, communications, and external trigger signal is documented in Figure 4.

Placeholder for Pin Mapping Diagram

Figure 4: Pin mapping for the OpenRB-150 and peripheral components.

3.11.8 Summary

The gripper actuation system leverages the built-in motion control capabilities of the Dynamixel AX-12A and a trigger-based switching mechanism to enable reliable, smooth gripping actions. The integration with the OpenRB-150 simplifies communication and provides modularity for future expansion, including closed-loop control or state-based task automation.

3.11.9 Custom Gripper - Installation and Operation

Installation Guide

[PLACEHOLDER: This section will provide detailed instructions for mounting the custom gripper onto the robot arm's flange. It will include a mechanical assembly guide with

diagrams, a bill of materials (BOM), and an electrical wiring diagram showing how to connect it to the control box or robot controller I/O.]

Standard Operating Procedure (SOP)

This SOP describes the routine use of the custom gripper.

1. Tool Center Point (TCP) Calibration:

- *[PLACEHOLDER: Step-by-step guide on how to perform a TCP calibration for the gripper. This is critical for accurate positioning. This will involve defining the TCP relative to the robot's flange using a 4-point or 6-point method via the DILBOT UI.]*

2. Gripping Operation:

- *[PLACEHOLDER: Instructions on how to use the 'Open Gripper' and 'Close Gripper' commands from the UI or within a recipe. Detail how to set parameters like gripping force and speed.]*

3. Maintenance and Cleaning:

- *[PLACEHOLDER: Guidelines for regular inspection, cleaning of gripper fingers to prevent contamination, and checking for mechanical wear.]*

3.12 Camera - Installation and Operation

Installation Guide

<https://support.cognex.com/en/downloads/detail/in-sight/4531/1033>

3.13 System Integration & Recipe Management

3.13.1 I/O pin distribution for both robots

Number of input pins on each robot = 16,

Number of output pins on each robot = 16,

Total number of input pins = 32,

Total number of output pins = 32,

Pin distribution table 1

Label	Ferrule No.	Signal Description	Device Connected To	Task
B1	21	24V	24VDC Power Distribution	Supply main power
B5	25	GPO 15	Backup Output	Spare control output
B6	26	GPO 14	Backup Output	Spare control output
B7	27	GPO 13	Spin Coater Motor	Speed control
B8	28	GPO 12	Spin Coater Motor	Speed control
B9	29	GPO 11	UV Ozone Tray Actuator	Open/close tray
B10	30	GPO 10	UV Ozone Lamp	On/Off control
B11	31	GPO 9	Input Loading Tray Motor	Tray movement
B12	32	GPO 8	Input Loading Tray Motor	Tray movement
B13	33	GPO 7	Gripper Solenoid	Grip/release
B14	34	GPO 6	Gripper Solenoid	Grip/release
B15	35	GPO 5	Motor Driver A	Wheel actuation
B16	36	GPO 4	Motor Driver A	Wheel actuation
B17	37	GPO 3	Motor Driver B	Wheel actuation
B18	38	GPO 2	Motor Driver B	Wheel actuation
B19	39	GPO 1	Buzzer / Indicator	Alert status
B20	40	GPO 0	System Enable Relay	Safety interlock
A1	1	0VDC	Power Distribution	Return reference
A2	2	0VDC	Sensor Ground	Signal return
A5	5	GPI 15	Distance Sensor Front	Obstacle detection
A6	6	GPI 14	Distance Sensor Side	Obstacle detection
A7	7	GPI 13	IMU Interrupt	Orientation feedback
A8	8	GPI 12	IMU Interrupt	Orientation feedback
A9	9	GPI 11	Encoder A	Speed/position feedback
A10	10	GPI 10	Encoder B	Speed/position feedback
A11	11	GPI 9	Gripper Feedback Switch	Grip detection
A12	12	GPI 8	Tray Home Switch	Position detection
A13	13	GPI 7	Tray Limit Switch	Position detection
A14	14	GPI 6	UV Tray Limit Switch	Safety feedback
A15	15	GPI 5	Spin Coater Home Switch	Position detection
A16	16	GPI 4	Spin Coater Limit Switch	Safety feedback
A17	17	GPI 3	Motor Overcurrent Sensor	Fault detection
A18	18	GPI 2	Emergency Stop	Safety interlock
A19	19	GPI 1	Start Button	Cycle initiation
A20	20	GPI 0	Stop Button	Cycle termination

Table 4: Pin Distribution with Device and Task Fields for robot 1

Label	Ferrule No.	Signal Description	Device Connected To	Task
B1	21	24V	24VDC Power Distribution	Supply main power
B5	25	GPO 15	Hamilton pipette 1	On or off
B6	26	GPO 14	Hamilton pipette 1	Wheel Actuation
B7	27	GPO 13	Hamilton pipette 1	Orientation Feedback
B8	28	GPO 12	Hamilton pipette 1	Object Handling
B9	29	GPO 11	Hamilton pipette 2	Obstacle Detection
B10	30	GPO 10	Hamilton pipette 2	On or off
B11	31	GPO 9	Hamilton pipette 2	Wheel Actuation
B12	32	GPO 8	Hamilton pipette 2	Orientation Feedback
B13	33	GPO 7	XY scanner	Object Handling
B14	34	GPO 6	XY scanner	Obstacle Detection
B15	35	GPO 5	XY scanner	On or off
B16	36	GPO 4	Vial loader	Wheel Actuation
B17	37	GPO 3	Vial loader	Orientation Feedback
B18	38	GPO 2	N2 flush	Object Handling
B19	39	GPO 1	Backup pins	Obstacle Detection
B20	40	GPO 0	Backup pins	Obstacle Detection
A1	1	0VDC	Power Distribution	Return reference
A2	2	0VDC	Sensor Ground	Signal return
A5	5	GPI 15	Hamilton pipette 1	On or off
A6	6	GPI 14	Hamilton pipette 1	Wheel Actuation
A7	7	GPI 13	Hamilton pipette 1	Orientation Feedback
A8	8	GPI 12	Hamilton pipette 1	Object Handling
A9	9	GPI 11	Hamilton pipette 2	Obstacle Detection
A10	10	GPI 10	Hamilton pipette 2	On or off
A11	11	GPI 9	Hamilton pipette 2	Wheel Actuation
A12	12	GPI 8	Hamilton pipette 2	Orientation Feedback
A13	13	GPI 7	Gripper Servo	Object Handling
A14	14	GPI 6	Distance Sensor Front	Obstacle Detection
A15	15	GPI 5	Gripper	On or off
A16	16	GPI 4	Motor Driver A	Wheel Actuation
A17	17	GPI 3	IMU Interrupt	Orientation Feedback
A18	18	GPI 2	Gripper Servo	Object Handling
A19	19	GPI 1	Distance Sensor Front	Obstacle Detection
A20	20	GPI 0	Distance Sensor Front	Obstacle Detection

Table 5: Pin Distribution with Device and Task Fields for robot 2

FR-SI-001 Centralized Control: The system shall provide a single, centralized control application.

FR-SI-002 Recipe Creation & Management: The system shall allow users to create, edit, save, load, and delete custom recipes.

FR-SI-003 Recipe Definition: Recipes shall be defined as a sequence of discrete operations. Examples:

```

1 Robot.MoveTo(Waypoint_Name)
2 Robot.PickUp(Substrate_Type, Source_Location)

```



```
3   SpinCoater.RunProgram(Program_ID)
4   Pipette.Aspirate(Volume, Source_Well_ID)
5   Gripper.Close(Force_Level)
6
```

FR-SI-004 Conditional Logic: The system shall support conditional logic within recipes.

FR-SI-005 Real-time Status: The system shall provide real-time status updates.

FR-SI-006 Data Logging: The system shall automatically log all executed actions to persistent storage.

FR-SI-007 Error Handling & Recovery: The system shall implement robust error handling mechanisms.

FR-SI-008 Calibration Routines: The system shall include software routines to assist with calibration of robot arm waypoints.

3.14 User Interface (UI)

FR-UI-001 Intuitive Recipe Editor: The UI shall provide a graphical recipe editor.

FR-UI-002 System Status Display: The UI shall display the current status of each module.

FR-UI-003 Visual Feedback: The UI shall provide visual feedback on robot position, gripper state, etc.

FR-UI-004 Emergency Stop Access: A prominent software emergency stop button shall be present on the UI.

FR-UI-005 Manual Control: The UI shall allow for manual control of individual modules.

FR-UI-006 Log Viewer: The UI shall include a dedicated section for viewing system logs.

FR-UI-007 User Authentication: The UI shall implement a basic user authentication system.

4 Non-Functional Requirements

4.1 Performance Requirements

NFR-PER-001 Robot Arm Positional Accuracy: ± 0.1 mm and angular accuracy of $\pm 0.1^\circ$.

NFR-PER-002 Spin Coater Speed Tolerance: ± 1 RPM tolerance.

NFR-PER-003 Pipetting Volume Accuracy: $\pm 1\%$ for volumes $> [10 \text{ }\mu\text{L}]$, and $\pm$ [Absolute error] for smaller volumes.

NFR-PER-004 Recipe Execution Time: Execute typical recipe within 15 minutes.

NFR-PER-005 Communication Latency: Shall not exceed 100 ms for critical cycles.

NFR-PER-006 Throughput: Capable of processing [Number] substrates per hour.

4.2 Safety Requirements

NFR-SAF-001 Physical Emergency Stop: At least one physical E-stop button.

NFR-SAF-002 Software Safety Interlocks: Prevent robot movement into exclusion zones, conflicting operations, etc.

NFR-SAF-003 Physical Guarding: All moving parts shall be physically guarded.

NFR-SAF-004 Visual Status Indicators: Clear visual indicators for system status.

NFR-SAF-005 Error State Handling: System shall transition to a safe, halted state on critical error.

NFR-SAF-006 Chemical Safety: System design shall consider compatibility with chemicals used.

4.3 Reliability Requirements

NFR-REL-001 Continuous Operation: At least 8 hours without manual intervention.

NFR-REL-002 Mean Time Between Failures (MTBF): At least 200 hours.

NFR-REL-003 Communication Robustness: Gracefully handle temporary communication loss.

NFR-REL-004 Recipe Repeatability: Execute the same recipe 100 times consecutively without system failure.

4.4 Maintainability Requirements

NFR-MNT-001 Modular Software Design: Highly modular software with well-defined APIs.

NFR-MNT-002 Component Accessibility: Custom hardware designed for easy assembly/disassembly.

NFR-MNT-003 Diagnostic Tools: Provide built-in diagnostic tools and detailed logs.

NFR-MNT-004 Calibration Procedures: Comprehensive and clear documentation shall be provided.

NFR-MNT-005 Software Updates: Support easy software updates and patches.

4.5 Usability Requirements

NFR-US-001 Recipe Creation Time: A new user can create a basic recipe within 30 minutes after 1-hour training.

NFR-US-002 Clear Error Messages: Error messages shall be clear, concise, and provide actionable information.

NFR-US-003 Feedback Consistency: Provide consistent visual and auditory feedback.

NFR-US-004 Intuitive Navigation: The UI navigation shall be intuitive.

NFR-US-005 Language Support: The UI shall primarily be in English.

4.6 Security Requirements

NFR-SEC-001 Access Control: Access protected by a multi-level user authentication system.

NFR-SEC-002 Software Updates: Third-party libraries shall be regularly updated.

NFR-SEC-003 Network Security: Network interfaces shall be configured to minimize exposure.

NFR-SEC-004 Data Protection: All logged data and recipes shall be stored securely.

5 System Architecture (High-Level)

5.1 Software Architecture

The software will follow a modular, layered architecture to ensure maintainability, scalability, and reusability.

Layer 1: Hardware Abstraction Layer (HAL) • **Purpose:** Provides a standardized interface to interact with each specific hardware module.

- **Components:** Robot Driver, Spin Coater Driver, Hamilton Pipette Driver, Gripper Driver.

Layer 2: Module Control Layer • **Purpose:** Exposes high-level, human-readable control functions for each module.

- **Components:** RobotController, SpinCoaterController, PipetteController, GripperController.

Layer 3: Recipe Execution Engine • **Purpose:** Interprets and executes user-defined recipes.

- **Components:** Recipe Parser, Sequence Manager, State Machine.

Layer 4: User Interface (UI) / HMI • **Purpose:** Provides the graphical interface for user interaction.

- **Components:** Recipe Editor, System Status Dashboard, Manual Control Panel, Log Viewer.

Cross-Cutting Concerns • **Data Logging System:** Centralized service for recording all system events.

- **Error Management System:** Centralized system for detecting and responding to errors.
- **Configuration Management:** Stores and manages system-wide settings.

5.2 Hardware Architecture

The hardware architecture consists of a central control unit connected to various laboratory automation modules.

Central Control PC

Robot Arm: Mitsubishi Electric RH-6CH-Sxx

Spin Coater: SCS G4 Spin Coater Series

Pipetting Module: Hamilton ZEUS X1

Custom Gripper

Safety System

Complementary Attachments & Labware

Power Distribution Unit

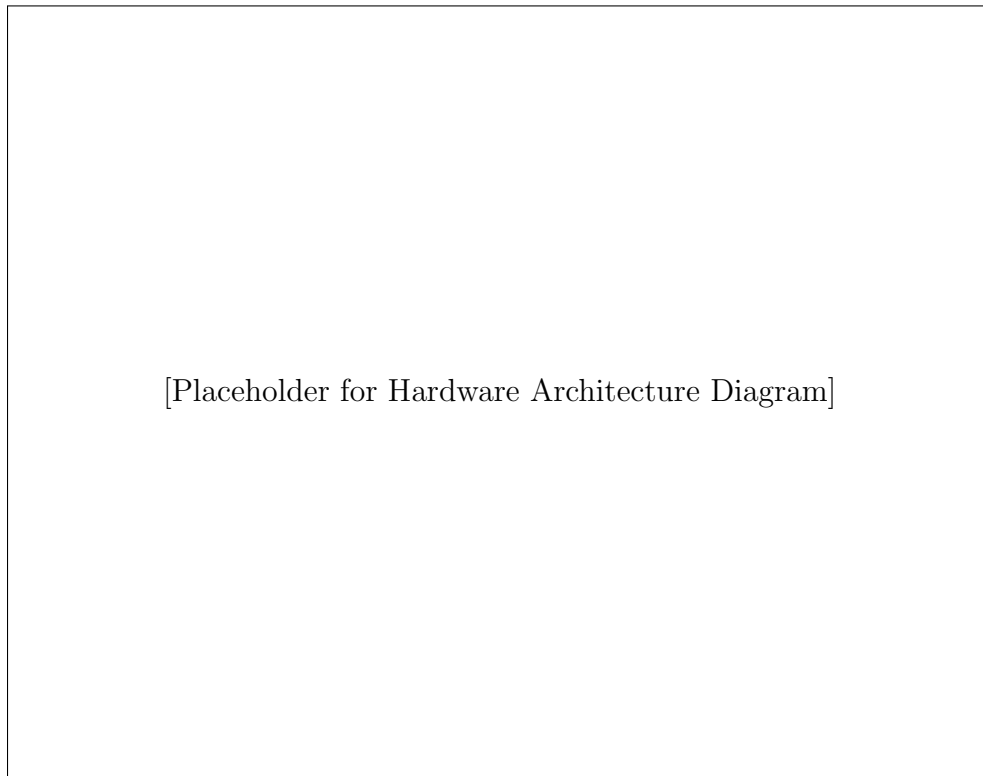


Figure 5: High-level diagram of the DILBOT hardware components.

6 Verification Requirements

VR-001 Functional Requirements Verification: All functional requirements (Section 3) shall be verified through a combination of Unit Testing, Integration Testing, System-Level Testing, and Test Cases.

VR-002 Performance Requirements Verification: Verified through Quantitative Measurements, Repeated Trials, and Benchmarking.

VR-003 Safety Requirements Verification: Verified through Safety Audits, Failure Mode and Effects Analysis (FMEA), Simulated Faults, and Emergency Stop Activation Tests.

VR-004 Reliability Requirements Verification: Verified through Endurance Testing, Stress Testing, and Error Log Analysis.

VR-005 Maintainability Requirements Verification: Verified through Code Reviews, Maintenance Drills, and Documentation Review.

VR-006 Usability Requirements Verification: Verified through User Acceptance Testing (UAT), Time-on-Task Measurements, and User Feedback Surveys.

VR-007 Security Requirements Verification: Verified through Access Control Testing and Vulnerability Scanning.

A Appendices

A.1 Appendix A: Detailed Communication Protocols

[PLACEHOLDER: This appendix will contain a comprehensive document outlining the specific commands, data formats, and expected responses for each integrated device (Mitsubishi Robot, SCS Spin Coater, Hamilton Pipette, Custom Gripper). It will include examples of command sequences and data parsing logic.]

A.2 Appendix B: CAD Drawings and Mechanical Schematics

[PLACEHOLDER: This appendix will contain detailed 2D and 3D CAD drawings of the integrated system layout, mechanical schematics for the custom gripper (including BOM and assembly instructions), and drawings for all complementary attachments.]

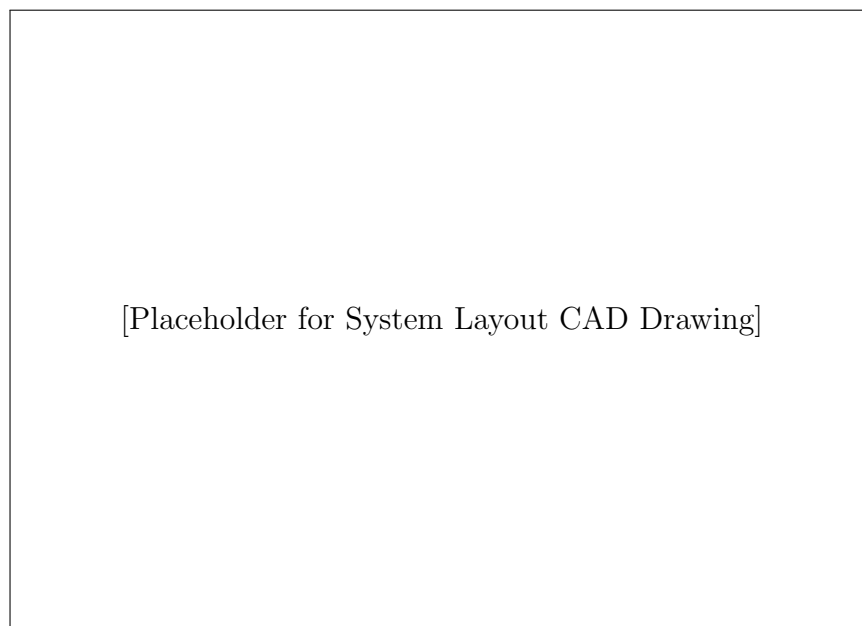


Figure 6: Example CAD drawing of the integrated system.

A.3 Appendix C: Electrical Schematics

[PLACEHOLDER: This appendix will provide detailed electrical schematics for the custom gripper's control board and wiring diagrams for power distribution and inter-module connections.]

A.4 Appendix D: Test Plans

[PLACEHOLDER: This appendix will house specific test plans for unit, integration, system, safety, and performance testing. It will include detailed test cases, expected results, and criteria for success for each verification requirement.]

A.5 Appendix E: Glossary

[PLACEHOLDER: This appendix will contain an expanded list of terms and definitions specific to the DILBOT project, beyond the abbreviations listed in the introduction.]